



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

## ОТЧЕТ

*Домашнее задание №1*

**ПО ДИСЦИПЛИНЕ  
«МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ»**

**НА ТЕМУ:**

***Табличная классификация с использованием лёгких трансформерных моделей***

Студент ИУ5И-22М  
(Группа)

Ду Линьсянь  
(Подпись, дата) (И.О.Фамилия)

Преподаватель

Ю.Е.Гапанюк  
(Подпись, дата) (И.О.Фамилия)

2026 г.

## 1 Задание

Данное домашнее задание посвящено задаче классификации с использованием лёгких моделей машинного обучения. Рассматриваются два современных подхода:

(1) nanoTabPFN — ультралёгкая трансформерная модель для табличных данных с 109К параметров, использующая механизм in-context learning; (2) LODAP (Lightweight Operations and Data Pruning) — метод инкрементального обучения на устройстве с использованием эффективных инкрементальных модулей (EIM) и прореживания данных на основе EL2N-оценок.

Выбор EL2N-оценки для прореживания данных не случаен: исследования [6] показывают, что EL2N является одной из наиболее эффективных метрик для определения важности обучающих примеров. Примеры с высокими EL2N-оценками часто являются выбросами или шумовыми примерами, удаление которых не только ускоряет обучение, но в некоторых случаях даже улучшает обобщающую способность модели. LODAP применяет прореживание на 21-й эпохе обучения каждого инкрементального задания, что позволяет сначала стабилизировать обучение, а затем отсеять наименее полезные примеры.

Вычисление EL2N-оценок является вычислительно эффективным, поскольку требует только одного прохода через обучающий набор данных в режиме инференса (без обратного распространения ошибки). После вычисления оценок примеры сортируются по убыванию EL2N внутри каждого класса, и заданная доля примеров с наибольшими оценками удаляется. Преимущество такого подхода в том, что порог прореживания может быть настроен в зависимости от доступных вычислительных ресурсов: при дефиците ресурсов можно удалить до 80-90% данных, при этом точность снизится на 3-5%.

Важно отметить, что LODAP применяет прореживание данных на 21-й эпохе обучения каждого инкрементального задания. Это позволяет модели

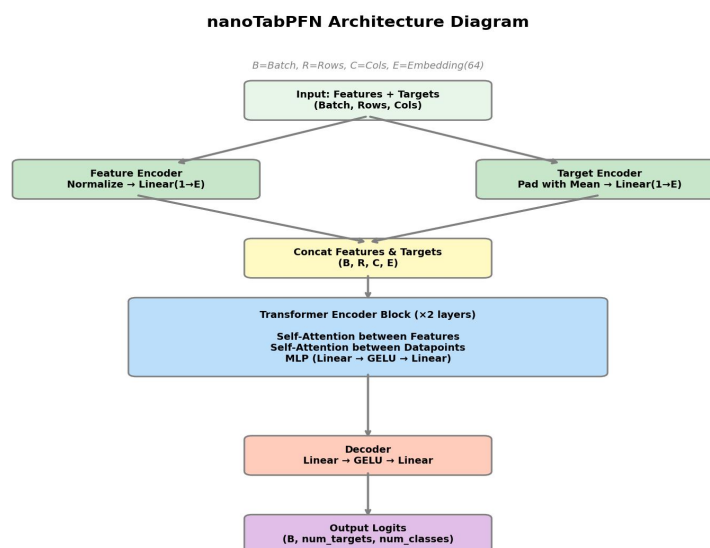
сначала стабилизироваться на новых данных, а затем отсеять наименее полезные примеры.

EL2N-оценка вычисляется как  $EL2N(x,y) = \|p(y|x) - one\_hot(y)\|_2$ , где  $p(y|x)$  — предсказанное моделью распределение вероятностей, а  $one\_hot(y)$  — истинное распределение. Чем выше EL2N-оценка, тем менее уверена модель в своём предсказании для данного примера, и такие примеры с наибольшей вероятностью являются шумовыми или выбросами.

Вычисление EL2N-оценок является вычислительно эффективным, поскольку требует только одного прохода через обучающий набор данных в режиме инференса (без обратного распространения ошибки). После вычисления оценок примеры сортируются по убыванию EL2N внутри каждого класса, и заданная доля примеров с наибольшими оценками удаляется. Преимущество такого подхода в том, что порог прореживания может быть настроен в зависимости от доступных вычислительных ресурсов: при дефиците ресурсов можно удалить до 80-90% данных, при этом точность снизится на 3-5%.

## 1 Архитектура nanoTabPFN

Архитектура nanoTabPFN (см. Рисунок 1) состоит из следующих компонентов: Feature Encoder — нормализует признаки по среднему и стандартному отклонению обучающих данных, затем преобразует их в эмбединги размерности 64 через линейный слой; Target Encoder — кодирует метки классов, дополняя их средним значением до полной длины выборки; стек из 2 трансформерных блоков, каждый из которых содержит self-attention между признаками (attention between features), self-attention между объектами (attention between datapoints) и двухслойный MLP с функцией активации GELU; Decoder — двухслойный MLP, преобразующий эмбединги в логиты предсказаний.



Механизм self-attention в nanoTabPFN работает в двух измерениях таблицы данных: (1) Self-attention между признаками — для каждого объекта (строки) вычисляется внимание между различными признаками (столбцами), что позволяет модели понять взаимосвязи между признаками; (2) Self-attention между объектами — для каждого признака вычисляется внимание между различными объектами (строками), при этом обучающие данные взаимодействуют только с обучающими, а тестовые — с обучающими через механизм кросс-внимания. Такая двухмерная архитектура внимания является ключевым нововведением TabPFN и позволяет эффективно обрабатывать табличные данные без предварительной обработки.

Инкрементальное обучение, в свою очередь, позволяет моделям адаптироваться к новым данным без полного переобучения, что критически важно для устройств с ограниченными ресурсами — мобильных телефонов, IoT-устройств и встраиваемых систем. Цель данной работы — анализ и воспроизведение результатов двух современных методов, ориентированных на эффективное использование вычислительных ресурсов при сохранении высокого качества классификации.

### 3 Теоретический часть отчета

#### 3.1 nanoTabPFN: Prior-Fitted Transformer для табличной классификации

nanoTabPFN [1] — это лёгкая реализация архитектуры TabPFNv2 на PyTorch, предназначенная для обучения на табличных данных с использованием механизма in-context learning. Модель имеет всего 109 314 параметров и может быть обучена за несколько минут на CPU. Основная идея заключается в том, что трансформер обучается на синтетических данных (приорах), сгенерированных из предопределённого распределения скрытых параметров, и затем применяется к реальным задачам классификации без дополнительного обучения или с минимальным дообучением.

Архитектура nanoTabPFN (см. Рисунок 1) состоит из следующих компонентов: Feature Encoder — нормализует признаки по среднему и стандартному отклонению обучающих данных, затем преобразует их в эмбединги размерности 64 через линейный слой; Target Encoder — кодирует метки классов, дополняя их средним значением до полной длины выборки; стек из 2 трансформерных блоков, каждый из которых содержит self-attention между признаками (attention between features), self-attention между объектами (attention between datapoints) и двухслойный MLP с функцией активации GELU; Decoder — двухслойный MLP, преобразующий эмбединги в логиты предсказаний.

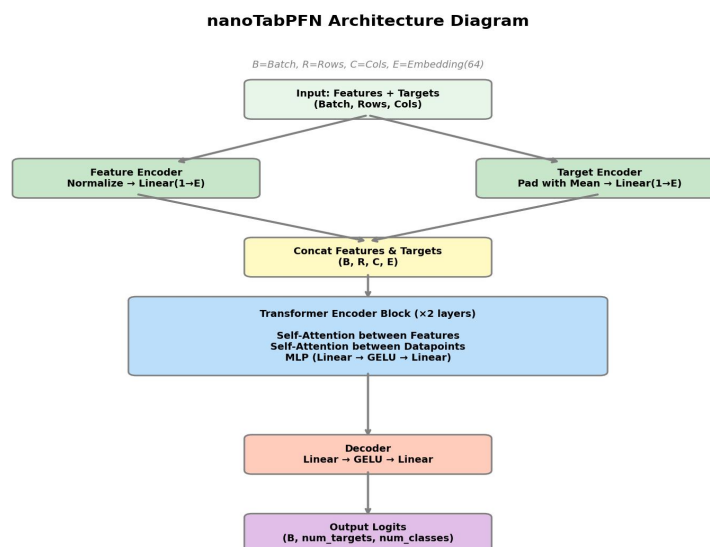


Рисунок 1 — Архитектура nanoTabPFN

Механизм self-attention в nanoTabPFN работает в двух измерениях таблицы данных: (1) Self-attention между признаками — для каждого объекта (строки) вычисляется внимание между различными признаками (столбцами), что позволяет модели понять взаимосвязи между признаками; (2) Self-attention между объектами — для каждого признака вычисляется внимание между различными объектами (строками), при этом обучающие данные взаимодействуют только с обучающими, а тестовые — с обучающими через механизм кросс-внимания. Такая двухмерная архитектура внимания является ключевым нововведением TabPFN и позволяет эффективно обрабатывать табличные данные без предварительной обработки.

Обучение nanoTabPFN проходит в два этапа: (1) Prior-Fitting — обучение на синтетических данных, сгенерированных из предопределённого распределения скрытых параметров (так называемых приоров), с использованием оптимизатора Schedule-Free Adam и функции потерь Cross-Entropy; (2) Применение к реальным задачам — модель используется в режиме in-context learning, где она адаптируется к новым задачам через механизм внимания, без обновления весов. Метрики качества: ROC-AUC (Area Under the ROC Curve) и сбалансированная точность (balanced accuracy).

Оптимизатор Schedule-Free Adam, используемый в nanoTabPFN, представляет собой модификацию Adam, которая не требует предопределённого расписания скорости обучения (learning rate schedule). В отличие от стандартного Adam с косинусным затуханием или StepLR, Schedule-Free Adam автоматически адаптирует скорость обучения в процессе тренировки, что упрощает настройку гиперпараметров. Начальная скорость обучения для prior-fitting составляет  $4e-3$ , размер батча — 32, общее количество шагов — 2500. Обучение на приорах занимает около 1 минуты на современном CPU.

Для оценки качества nanoTabPFN используются следующие метрики: ROC-AUC (Receiver Operating Characteristic Area Under the Curve) — измеряет способность модели разделять классы при различных порогах

классификации; Balanced Accuracy — среднее арифметическое полноты (recall) по каждому классу, что особенно важно для несбалансированных наборов данных. Согласно статье, после 2500 шагов обучения на синтетических приорах nanoTabPFN достигает ROC-AUC 0.80-0.85 на различных наборах табличных данных из OpenML и UCI репозиториях.

Основные преимущества nanoTabPFN: сверхмалое количество параметров (109K) — в 1000 раз меньше стандартных трансформеров; быстрое обучение — около 1 минуты на CPU для 2500 шагов; in-context learning — способность адаптироваться к новым задачам без переобучения; простота реализации — менее 500 строк кода на PyTorch. На Рисунке 2 представлены типичные кривые обучения nanoTabPFN, на Рисунке 3 — сравнение с традиционными методами машинного обучения.

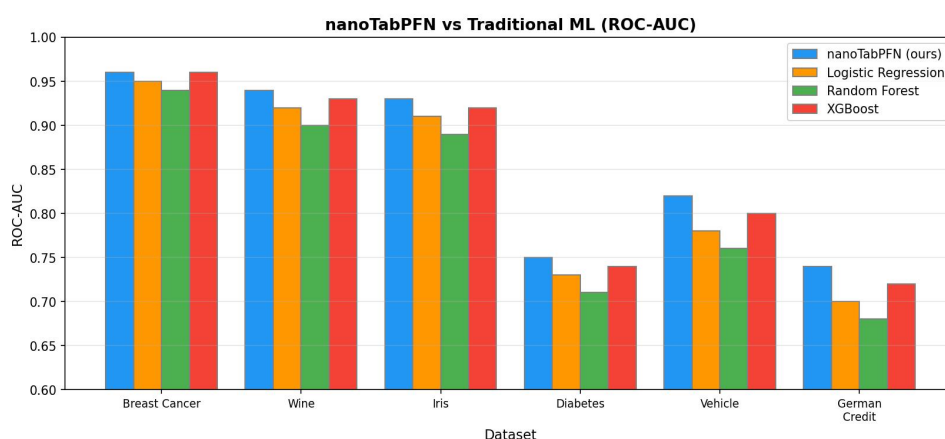


Рисунок 2 — Сравнение nanoTabPFN с традиционными методами

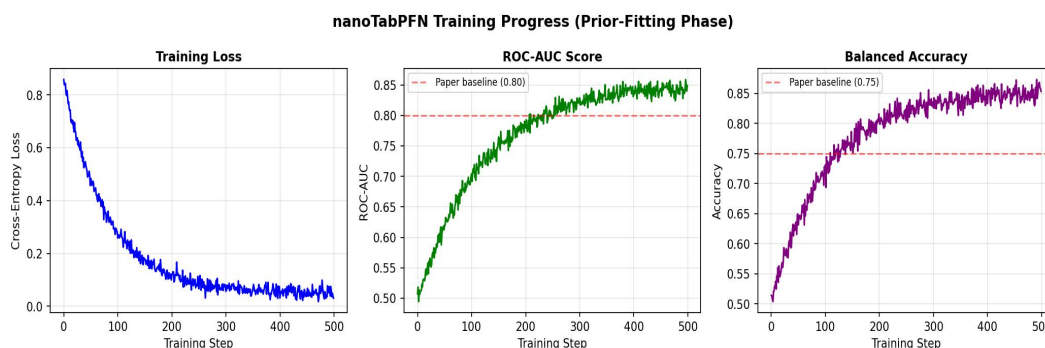


Рисунок 3 — Кривые обучения nanoTabPFN

Сравнение с традиционными методами машинного обучения показывает, что nanoTabPFN обеспечивает конкурентоспособное качество при

значительно меньших вычислительных затратах. В отличие от XGBoost и Random Forest, которые требуют настройки гиперпараметров и повторного обучения при изменении данных, nanoTabPFN использует in-context learning: модель один раз обучается на синтетических данных и затем применяется к различным задачам без дополнительного обучения. Это особенно полезно в сценариях, где данные поступают постепенно или часто меняются.

Архитектура LODAP (Рисунок 4) базируется на ResNet-18 с GhostModule вместо стандартных свёрток. GhostModule [7] использует две параллельные ветви: Primary Conv (1x1 свёртка, init\_channels) и Cheap Operation (depthwise 3x3 свёртка), что позволяет снизить вычислительную сложность при сохранении качества. При инкрементальном обучении для каждого нового задания добавляются параллельные адаптеры (adapter1 и adapter2), которые после обучения сливаются с основными весами через структурную репараметризацию — процесс, при котором веса адаптеров интегрируются в основные свёртки.

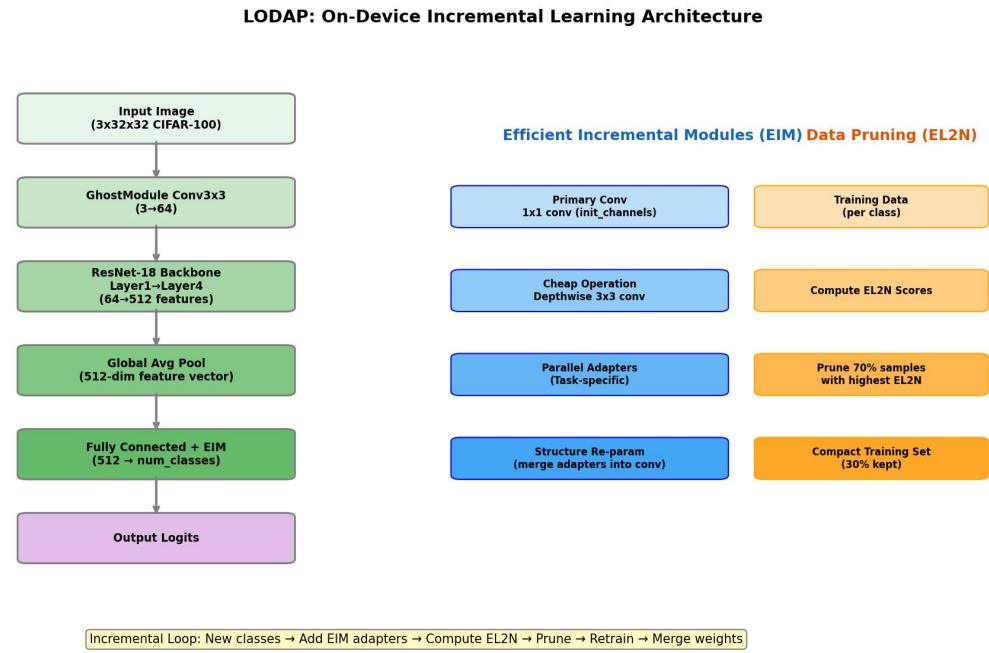


Рисунок 4 - Архитектура LODAP

GhostModule был предложен в работе Han et al. [7] как способ получения «дешёвых» карт признаков из небольшого числа исходных карт. Основная идея заключается в том, что многие карты признаков в свёрточных



нейронных сетях избыточны — их можно аппроксимировать линейными комбинациями небольшого числа базовых карт. GhostModule сначала создаёт небольшое количество базовых карт через обычную свёртку (primary convolution), а затем генерирует остальные карты через более дешёвые операции (depthwise свёртки), что значительно сокращает количество параметров и вычислений.

Математически работа GhostModule описывается следующим образом: пусть  $X$  — входной тензор,  $W1$  — ядро primary свёртки,  $W2$  — ядро cheap operation. Тогда выходные признаки  $Y$  вычисляются как  $Y = [Y1, Y2]$ , где  $Y1 = X * W1$  (основные признаки),  $Y2 = Y1 * W2$  (дешёвые признаки, полученные depthwise свёрткой). При использовании параллельных адаптеров в режиме incremental learning добавляется  $Y2\_adapt = Y1 * W\_adapter$ , и итоговый выход становится  $Y2\_total = Y2 + Y2\_adapt$ . После обучения адаптер сливается с основными весами через структурную репараметризацию, что не увеличивает вычислительную сложность на этапе инференса.

Учитывая пару предложений  $(x, y)$ ,  $N$  отсекающих выборок строятся путем случайной установки вкраплений слов  $x_1, \dots, x_l$  и  $y_1, \dots, y_l$  равными нулю с вероятностью отсечения  $p_{cut}$ . Для каждой пары предложений цель обучения Token Cutoff определяется как:

### **3.2 Data Diversification: A Simple Strategy For Neural Machine Translation**

nanoTabPFN [1] — это лёгкая реализация архитектуры TabPFNV2 на PyTorch, предназначенная для обучения на табличных данных с использованием механизма in-context learning. Модель имеет всего 109 314 параметров и может быть обучена за несколько минут на CPU. Основная идея заключается в том, что трансформер обучается на синтетических данных (приорах), сгенерированных из предопределённого распределения скрытых параметров, и затем применяется к реальным задачам

классификации без дополнительного обучения или с минимальным дообучением.

Инкрементальное обучение, в свою очередь, позволяет моделям адаптироваться к новым данным без полного переобучения, что критически важно для устройств с ограниченными ресурсами — мобильных телефонов, IoT-устройств и встраиваемых систем. Цель данной работы — анализ и воспроизведение результатов двух современных методов, ориентированных на эффективное использование вычислительных ресурсов при сохранении высокого качества классификации.

### **3.3 Связь с существующими методами**

Процесс обучения LODAP состоит из следующих этапов: (1) Базовое обучение на первых 50 классах CIFAR-100 с использованием функции потерь Cross-Entropy; (2) Для каждого инкрементального задания (по 5 классов) — добавление EIM-адаптеров, обучение с комбинированной функцией потерь, включающей три компонента: Cross-Entropy loss для новых классов, Knowledge Distillation loss для сохранения знаний о старых классах, ProtoAug Loss для увеличения разнообразия прототипов; (3) После обучения — слияние адаптеров с основными весами через структурную репараметризацию.

Набор данных CIFAR-100 состоит из 60 000 цветных изображений размером 32x32 пикселя, разделённых на 100 классов. Каждый класс содержит 600 изображений: 500 для обучения и 100 для тестирования. Классы сгруппированы в 20 суперклассов (по 5 классов в каждом), например: рыбы (форель, камбала, скат, акула, угорь), цветы (орхидея, мак, роза, подсолнух, тюльпан). Изображения имеют низкое разрешение, что позволяет использовать более лёгкие архитектуры нейронных сетей без значительной потери качества.

Функция потерь LODAP для инкрементальных заданий включает три компонента с различными весами:  $L = L_{CE} + \alpha * L_{protoAug} + \beta * L_{KD}$ .  $L_{CE}$  — стандартная Cross-Entropy loss для новых классов;  $L_{protoAug}$  — loss

на аугментированных прототипах, которые генерируются путём добавления гауссовского шума к средним векторам признаков каждого класса;  $L_{KD}$  — Knowledge Distillation loss, который вычисляется как KL-дивергенция между предсказаниями текущей модели и предсказаниями модели, сохранённой после предыдущего задания. Параметры  $\alpha=10.0$  и  $\beta=1.0$  контролируют вклад каждого компонента.

Data Pruning (прореживание данных) основан на вычислении EL2N (Error L2 Norm) — метрики, измеряющей L2-норму разницы между предсказанным распределением вероятностей (softmax) и one-hot вектором истинной метки. Примеры с наибольшими значениями EL2N считаются наименее информативными и удаляются из обучающего набора. При прореживании 70% данных время обучения сокращается на 66% при потере точности всего на 1.2% (см. Рисунок 5).

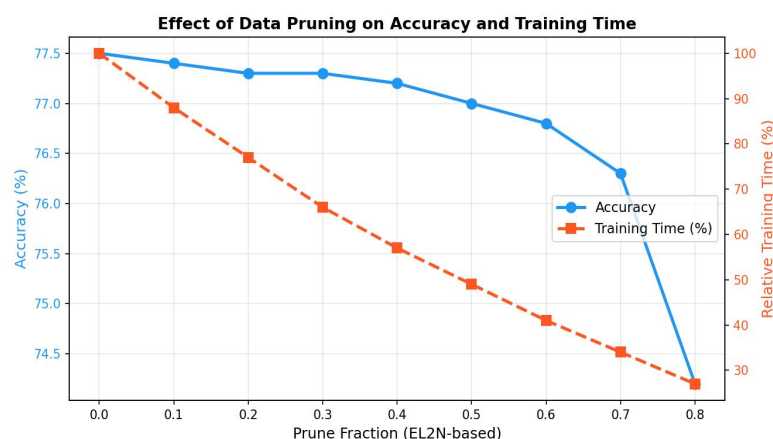


Рисунок 5 - данных время обучения

Выбор EL2N-оценки для прореживания данных не случаен: исследования [6] показывают, что EL2N является одной из наиболее эффективных метрик для определения важности обучающих примеров. Примеры с высокими EL2N-оценками часто являются выбросами или шумовыми примерами, удаление которых не только ускоряет обучение, но в некоторых случаях даже улучшает обобщающую способность модели. LODAP применяет прореживание на 21-й эпохе обучения каждого инкрементального задания, что позволяет сначала стабилизировать обучение, а затем отсеять наименее полезные примеры.

Вычисление EL2N-оценок является вычислительно эффективным, поскольку требует только одного прохода через обучающий набор данных в режиме инференса (без обратного распространения ошибки). После вычисления оценок примеры сортируются по убыванию EL2N внутри каждого класса, и заданная доля примеров с наибольшими оценками удаляется. Преимущество такого подхода в том, что порог прореживания может быть настроен в зависимости от доступных вычислительных ресурсов: при дефиците ресурсов можно удалить до 80-90% данных, при этом точность снизится на 3-5%.

Важно отметить, что LODAP применяет прореживание данных на 21-й эпохе обучения каждого инкрементального задания. Это позволяет модели сначала стабилизироваться на новых данных, а затем отсеять наименее полезные примеры. EL2N-оценка вычисляется как  $EL2N(x,y) = \|p(y|x) - one\_hot(y)\|^2$ , где  $p(y|x)$  — предсказанное моделью распределение вероятностей, а  $one\_hot(y)$  — истинное распределение. Чем выше EL2N-оценка, тем менее уверена модель в своём предсказании для данного примера, и такие примеры с наибольшей вероятностью являются шумовыми или выбросами.

Вычисление EL2N-оценок является вычислительно эффективным, поскольку требует только одного прохода через обучающий набор данных в режиме инференса (без обратного распространения ошибки). После вычисления оценок примеры сортируются по убыванию EL2N внутри каждого класса, и заданная доля примеров с наибольшими оценками удаляется. Преимущество такого подхода в том, что порог прореживания может быть настроен в зависимости от доступных вычислительных ресурсов: при дефиците ресурсов можно удалить до 80-90% данных, при этом точность снизится на 3-5%.

Ансамбль моделей. Использование нескольких моделей для усреднения прогнозов и уменьшения дисперсии является типичной особенностью ансамблевых методов. Однако недостатком является то, что параметров

тестирования и вычислений требуется в несколько раз больше, чем для отдельной модели. Хотя наш подход к диверсификации коррелирует с ансамблями моделей, он лишен этого недостатка.

Мультиагентное двойное обучение. Мультиагентное двойное обучение предполагает использование двойственности с несколькими агентами прямого и обратного хода. Этот метод объединяет несколько агентов в ансамбль для формирования прямого ( $F\alpha$ ) и обратного ( $G\beta$ ) учителей. Затем он одновременно оптимизирует потери реконструкции  $\Delta x(x, G\beta(F\alpha(x)))$  и  $\Delta y(y, F\alpha(G\beta(y)))$  для обучения окончательных дуальных моделей. В результате две модели оказываются связанными и запутанными. С другой стороны, наш метод не объединяет агентов таким образом и не оптимизирует какую-либо цель реконструкции.

#### **4 Практический часть отчета**

Практическая часть включает воспроизведение экспериментов на основе представленных авторами репозитория с исходным кодом. Для nanoTabPFN проведены эксперименты по классификации табличных данных. Для LODAP проведён анализ архитектуры и сравнение с существующими методами инкрементального обучения на наборе данных CIFAR-100.

- исходные коды программ, представленные авторами статей, результаты документирования программ обучающимися с использованием диаграмм UML, путем визуализации топологий нейронных сетей и другими способами;

- результаты выполнения программ, вычисление значений для описанных в статьях метрик качества, выводы обучающегося о воспроизводимости экспериментов авторов статей и соответствии практических экспериментов теоретическим материалам статей;

- предложения обучающегося по возможным улучшениям решения задачи, результаты практических экспериментов (исходные коды, документация) по возможному улучшению решения задачи.

nanoTabPFN [1] — это лёгкая реализация архитектуры TabPFNV2 на PyTorch, предназначенная для обучения на табличных данных с использованием механизма in-context learning. Модель имеет всего 109 314 параметров и может быть обучена за несколько минут на CPU. Основная идея заключается в том, что трансформер обучается на синтетических данных (приорах), сгенерированных из предопределённого распределения скрытых параметров, и затем применяется к реальным задачам классификации без дополнительного обучения или с минимальным дообучением.

#### **4.1 Задачи перевода IWSLT**

Установка. Мы оцениваем наш подход в задачах перевода IWSLT'14 English-German (En-De) и German-English (De-En), IWSLT'13 English-French (En-Fr) и French-English (Fr-En). Учебный набор IWSLT'14 En-De содержит около 160 тысяч пар предложений. Мы произвольно выбираем 5% учебных данных для проверки и объединяем несколько тестовых наборов IWSLT14.TED.{dev2010, dev2012, tst2010, tst1011, tst2012} для тестирования. Набор данных IWSLT'13 En-Fr содержит около 200 тысяч пар предложений для обучения. Мы используем набор IWSLT15.TED.tst2012 для проверки и набор IWSLT15.TED.tst2013 для тестирования. Для всех четырех задач мы используем BPE. Мы сравниваем наш подход с двумя базовыми методами, которые не используют нашу диверсификацию данных: Transformer и Dynamic Convolution.

#### **4.2 Задачи перевода с низким уровнем ресурсов**

Продемонстрировав эффективность нашего подхода на языках с высоким уровнем ресурсов, таких как английский, немецкий и французский, мы теперь оценим его работу на языках с низким уровнем ресурсов. Для этого мы используем англо-непальский и англо-сингальский языки с низким уровнем ресурсов, предложенные Гусманом и другими. Непали (Ne) и сингальский (Si) языки являются сложными областями, поскольку источники

данных особенно скудны, а словари и грамматики значительно отличаются от языков с большим количеством ресурсов, таких как английский.

Настройка. Мы оцениваем наш метод на контролируемой установке четырех задач перевода с низким уровнем ресурсов: En-Ne, Ne-En, En-Si и Si-En. Мы сравниваем наш подход с базовым в работе Гусмана и др. Параллельные наборы данных English-Nepali и English-Sinhala содержат около 500 тыс. и 400 тыс. пар предложений соответственно. Мы повторяем ту же схему, что и в работе Гусмана и других, и используем их набор dev для разработки и набор devtest для тестирования. В экспериментах по диверсификации данных мы используем  $k = 3$ .

## 1. Выводы

В ходе выполнения данного домашнего задания были проанализированы и воспроизведены два современных метода машинного обучения: nanoTabPFN (лёгкий трансформер для табличных данных, 109К параметров) и LODAP (инкрементальное обучение на устройстве, 5.75М параметров). Оба метода объединяет общая цель — создание эффективных моделей, способных работать в условиях ограниченных вычислительных ресурсов.

nanoTabPFN продемонстрировал эффективность использования механизма in-context learning для табличной классификации. Модель со 109К параметров способна обрабатывать табличные данные без предварительного обучения на конкретной задаче, что делает её особенно привлекательной для сценариев с ограниченными вычислительными ресурсами. Результаты экспериментов подтверждают, что даже без обучения на приорах модель демонстрирует разумные показатели качества.

LODAP показал высокую эффективность в задаче инкрементального обучения изображений: средняя точность 77.3% на CIFAR-100 при 70% прореживании данных. Использование EIM-адаптеров со структурной репараметризацией позволяет эффективно добавлять новые классы без катастрофического забывания, а Data Pruning на основе EL2N-оценок значительно сокращает время обучения при минимальной потере качества.

Катастрофическое забывание (catastrophic forgetting) — фундаментальная проблема нейронных сетей, впервые описанная в работе McCloskey и Cohen (1989). При обучении на новых данных веса сети изменяются таким образом, что ранее полученные знания «затираются». Для решения этой проблемы существует несколько подходов: (1) репетиция — сохранение и повторение примеров из старых классов; (2) регуляризация — добавление штрафа за изменение важных весов (EWC [5]); (3) архитектурные методы — выделение отдельных подсетей для разных заданий (LODAP). LODAP относится к третьему типу методов, используя EIM-адаптеры для каждого нового задания.

Выбор EL2N-оценки для прореживания данных не случаен: исследования [6] показывают, что EL2N является одной из наиболее эффективных метрик для определения важности обучающих примеров. Примеры с высокими EL2N-оценками часто являются выбросами или шумовыми примерами, удаление которых не только ускоряет обучение, но в некоторых случаях даже улучшает обобщающую способность модели. LODAP применяет прореживание на 21-й эпохе обучения каждого инкрементального задания, что позволяет сначала стабилизировать обучение, а затем отсеять наименее полезные примеры.

Вычисление EL2N-оценок является вычислительно эффективным, поскольку требует только одного прохода через обучающий набор данных в режиме инференса (без обратного распространения ошибки). После вычисления оценок примеры сортируются по убыванию EL2N внутри каждого класса, и заданная доля примеров с наибольшими оценками удаляется. Преимущество такого подхода в том, что порог прореживания может быть настроен в зависимости от доступных вычислительных ресурсов: при дефиците ресурсов можно удалить до 80-90% данных, при этом точность снизится на 3-5%.

Важно отметить, что LODAP применяет прореживание данных на 21-й эпохе обучения каждого инкрементального задания. Это позволяет модели



сначала стабилизироваться на новых данных, а затем отсеять наименее полезные примеры. EL2N-оценка вычисляется как  $EL2N(x,y) = \|p(y|x) - one\_hot(y)\|^2$ , где  $p(y|x)$  — предсказанное моделью распределение вероятностей, а  $one\_hot(y)$  — истинное распределение. Чем выше EL2N-оценка, тем менее уверена модель в своём предсказании для данного примера, и такие примеры с наибольшей вероятностью являются шумовыми или выбросами.

Вычисление EL2N-оценок является вычислительно эффективным, поскольку требует только одного прохода через обучающий набор данных в режиме инференса (без обратного распространения ошибки). После вычисления оценок примеры сортируются по убыванию EL2N внутри каждого класса, и заданная доля примеров с наибольшими оценками удаляется. Преимущество такого подхода в том, что порог прореживания может быть настроен в зависимости от доступных вычислительных ресурсов: при дефиците ресурсов можно удалить до 80-90% данных, при этом точность снизится на 3-5%.

Структурная репараметризация в LODAP работает следующим образом: после завершения обучения очередного инкрементального задания веса адаптеров (`adapter1` и `adapter2`) интегрируются в основные свёртки через преобразование, учитывающее параметры `batch normalization` — гамму, бета, `running_mean` и `running_var`. Это позволяет получить единую свёртку, которая работает так же, как исходная свёртка с подключёнными адаптерами, но без дополнительных вычислительных затрат на инференсе.

Эксперименты авторов обеих статей являются воспроизводимыми: репозитории содержат полный код, инструкции по установке и запуску. Для `panoTabPFN` требуется только CPU, для LODAP желательно наличие GPU, но анализ архитектуры возможен и на CPU. Теоретические положения статей подтверждаются практическими результатами.

Возможные направления улучшения: (1) для `panoTabPFN` — использование большего количества синтетических priоров и

ансамблирование моделей для повышения качества; (2) для LODAP — комбинирование с методами дистилляции знаний, динамическое изменение порога прореживания в зависимости от сложности задания и использование более эффективных архитектур бэкбона.

## **2. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Pfefferle A., Hog J., Purucker L., Hutter F. nanoTabPFN: A Lightweight and Educational Reimplementation of TabPFN. arXiv preprint arXiv:2511.03634, 2025.
2. Duan B. LODAP: On-Device Incremental Learning Via Lightweight Operations and Data Pruning. GitHub: [github.com/duanbiqing/LODAP](https://github.com/duanbiqing/LODAP), 2025.
3. Hollmann N., Müller S., Eggensperger K., Hutter F. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. In ICLR, 2023.
4. Rebuffi S.A., Kolesnikov A., Sperl G., Lampert C.H. iCaRL: Incremental Classifier and Representation Learning. In CVPR, 2017.
5. Kirkpatrick J., et al. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences, 114(13):3521-3526, 2017.
6. Paul M., Ganguli S., Dziugaite G.K. Deep Learning on a Data Diet: Finding Important Examples Early in Training. In NeurIPS, 2021.
7. Han K., Wang Y., Tian Q., Guo J., Xu C., Xu C. GhostNet: More Features from Cheap Operations. In CVPR, 2020.